Project - High Level Design

on

Dockerized Entertainment React Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AMAN SHARMA	EN22CS301109
2	ANANYA BHATIA	EN22CS301117
3	ANKIT NAGAR	EN22CS301134
4	ANUSHA SINGH PANWAR	EN22CS301179
5	ARPIT PATIDAR	EN22CS301199

Group Name: Group 02D2

Project Number: DO-14

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: Jan-June 2026

Table of Contents

1. Introduction

The **Dockerized Entertainment React Service** is a deployment-oriented extension of the **Entertainment Content Generator**, a Generative AI-based system developed to automate the creation of structured entertainment content. The application enables users to transform a simple idea into comprehensive outputs such as concepts, loglines, story outlines, character profiles, and scene descriptions through the use of advanced Large Language Models (LLMs).

While the core functionality of the system is powered by Generative AI, the primary focus of this phase is to address a critical real-world challenge: **reliable and consistent deployment of AI applications**. To achieve this, the entire system has been containerized using Docker, encapsulating all components—including the user interface, content generation pipeline, AI integration, and vector-based memory—into a unified and portable environment.

This approach eliminates common issues related to dependency management, environment configuration, and system compatibility, ensuring that the application can be deployed and executed seamlessly across different platforms. By doing so, the project moves beyond a prototype-level implementation and demonstrates how AI-driven systems can be prepared for practical, production-like environments.

The system integrates:

- **Streamlit** for an interactive and intuitive user interface
- **Generative AI (LLMs)** for intelligent content generation
- **FAISS vector database** for maintaining contextual consistency
- **Docker** for containerization and deployment

1.1 Scope of the Document

This document presents the **High-Level Design (HLD)** of the **Dockerized Entertainment React Service**, focusing on the architectural structure and deployment strategy of the system. It outlines how the existing Generative AI-based application is packaged, organized, and deployed using containerization technologies.

The scope of this document includes:

- Description of the overall system architecture and its key components
- Design of the application layers, including the user interface, content generation pipeline, and AI integration
- Interaction between system components and external AI services
- Process flow and information flow within the application
- Docker-based containerization approach for consistent deployment
- Overview of data handling mechanisms and API interactions
- Key design considerations such as scalability, maintainability, and deployment efficiency
- High-level non-functional aspects including security and performance

1.2 Intended Audience

The primary audience includes:

- **Project Evaluators / Academic Faculty:**
  To assess the system architecture, design approach, and the integration of Generative AI with containerization technologies for academic evaluation.
- **Developers and Project Team Members:**
  To understand the overall system structure, component interactions, application flow, and implementation details for development and future enhancements.
- **DevOps / Deployment Engineers:**
  To gain insights into the Docker-based containerization approach, environment setup, and deployment strategy of the application.
- **Future Maintainers:**
  To serve as a reference for understanding the existing system design, enabling easier debugging, scaling, and feature additions.
- **Technical Stakeholders / Reviewers:**
  To evaluate the architectural decisions, modular design, scalability, and real-world applicability of the solution.

1.3 System Overview

The **Dockerized Entertainment React Service** is a containerized Generative AI-based application that enables users to generate structured entertainment content from a simple input idea. The system builds upon the existing **Entertainment Content Generator** by focusing on making the application **portable, consistent, and deployment-ready** using Docker.

The application provides a guided content generation workflow where users can input parameters such as idea, genre, and tone, and receive structured outputs including concepts, loglines, story outlines, character descriptions, and scene content. This is achieved through the integration of a **Large Language Model (LLM)**, which processes user input and generates context-aware content.

A key feature of the system is the use of a **semantic memory mechanism** powered by a vector database, which stores previously generated content as embeddings. This allows the system to maintain contextual continuity across different stages of content generation, ensuring coherence and consistency in the output.

From an architectural perspective, the system follows a layered design consisting of:

- **Presentation Layer:** A Streamlit-based user interface that handles user interaction and displays generated results
- **Application Layer:** A content pipeline that manages the execution of different generation stages
- **AI Layer:** Integration with a Generative AI model for intelligent content creation
- **Memory Layer:** A FAISS-based vector store for embedding storage and context retrieval
- **Deployment Layer:** Docker containerization that encapsulates the entire system for consistent and reliable execution

2. System Design

The system design of the **Dockerized Entertainment React Service** focuses on organizing the existing Generative AI application into a **modular, layered architecture** while enabling seamless deployment through Docker containerization. The design ensures clear separation of concerns between user interaction, content generation logic, AI integration, and deployment environment.

The system follows a **layered architecture**, where each layer performs a specific function and interacts with others in a controlled manner. This improves maintainability, scalability, and overall system clarity.

2.1 Application Design

The application is composed of multiple logical components that work together to deliver a structured and intelligent content generation experience. While the core functionality remains based on the existing GenAI pipeline, the design emphasizes **efficient execution and containerized deployment**.

2.1.1 Presentation Layer (Streamlit UI)

Responsibilities:

- Accept user inputs such as idea, genre, tone, and stage selection
- Display generated content in a structured format
- Provide options for full pipeline execution or stage-wise generation
- Enable regeneration of specific outputs
- Manage session state for user interaction

The frontend serves as the entry point for users and ensures a smooth and interactive experience without directly interacting with the AI or memory layers.

2.1.2 Application Layer (Content Pipeline)

Responsibilities:

- Orchestrate the execution of different content generation stages
- Construct structured prompts using user input and contextual memory
- Control workflow through functions such as stage-wise and full pipeline execution
- Coordinate between the UI, AI layer, and memory layer

This layer acts as the **core controller** of the system and ensures that all operations follow a structured and logical flow.

2.1.3 AI Integration Layer (LLM Client)

Responsibilities:

- Communicate with the Generative AI model (e.g., Gemini API)
- Apply prompt constraints and formatting rules
- Handle API requests, retries, and response processing
- Return structured AI-generated content

This layer abstracts the interaction with external AI services, ensuring flexibility and loose coupling with the application logic.

2.1.4 Memory Layer (Vector Store - FAISS)

Responsibilities:

- Convert generated content into vector embeddings
- Store embeddings along with metadata (stage, content ID, timestamp)
- Retrieve top-k relevant context for maintaining continuity
- Support semantic search for improved content coherence

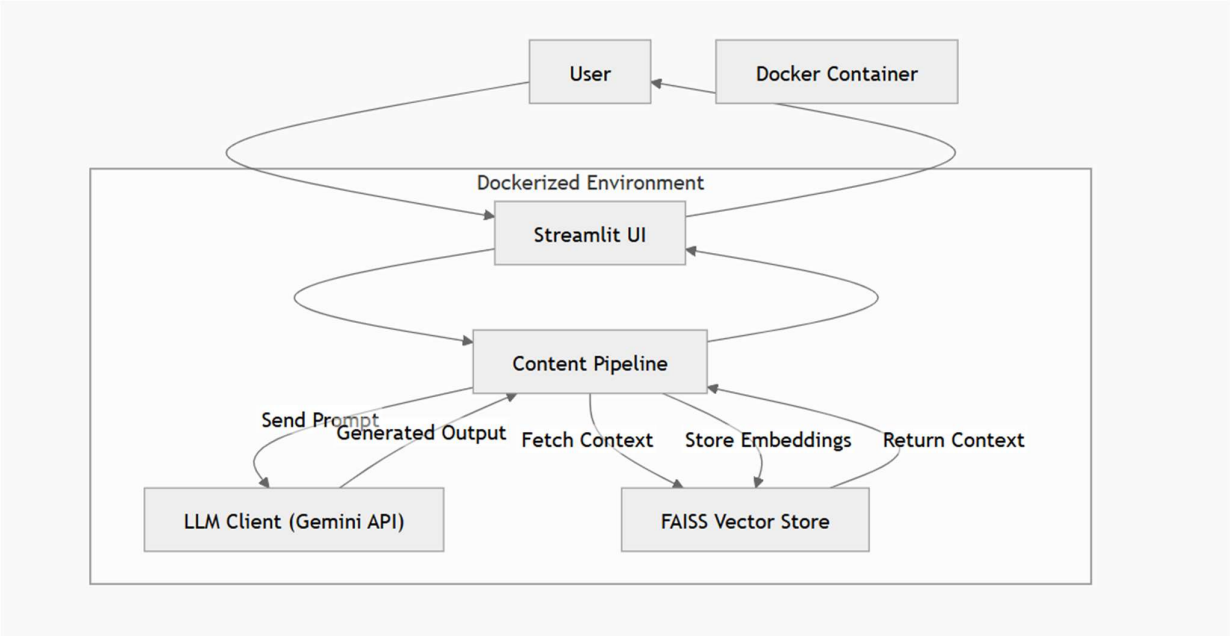
This layer ensures that the system maintains **context-aware generation across multiple stages**.

2.1.5 Deployment Layer (Docker Container)

Responsibilities:

- Package the entire application along with dependencies
- Provide an isolated and consistent runtime environment
- Enable easy deployment across different systems
- Eliminate environment-specific issues

The Docker container encapsulates all layers of the application, ensuring that the system runs uniformly regardless of the underlying infrastructure.



2.2 Process Flow

Steps:

1. **User Interaction**: The process begins when the user interacts with the application through the **Streamlit-based user interface**, providing inputs such as a seed idea, genre, tone, or selecting a specific content generation stage.

2. **Input Capture and Validation**: The user input is captured by the UI and validated to ensure it meets the required format before being forwarded for processing.

3. **Request to Content Pipeline**
   The validated input is sent to the **Content Pipeline (Application Layer)**, which acts as the central controller of the system.

4. **Context Retrieval from Memory**
   The pipeline queries the **FAISS vector store** to retrieve top-k semantically similar content from previous stages.
   This step ensures **contextual continuity** in content generation.

5. **Prompt Construction**: The pipeline constructs a structured prompt by combining:
    - User input
    - Retrieved contextual memory
    - Stage-specific instructions
    - System-level constraints

6. **LLM Invocation**: The constructed prompt is sent to the **LLM Client**, which forwards it to the **Generative AI model (e.g., Google Gemini API)**.

7. **Content Generation**: The AI model processes the prompt and generates structured output corresponding to the requested stage (e.g., concept, logline, outline, etc.).

8. **Response Processing**: The generated content is received by the pipeline and processed into a structured format suitable for display and storage.

9. **Embedding Generation and Storage**: The generated output is converted into vector embeddings using a **Sentence Transformer model** and stored in the **FAISS vector database** along with metadata.

10. **State Update and UI Rendering**: The processed output is sent back to the UI, where the application state is updated and the generated content is displayed to the user dynamically.

11. **Containerized Execution**: All the above steps are executed within a **Docker container**, ensuring:

- Consistent runtime environment
- Dependency isolation
- Easy deployment across systems

2.3 Information Flow

The system follows a well-defined information flow to ensure efficient data movement, contextual consistency, and seamless interaction between different components. The flow is designed to support **multi-stage content generation** while maintaining coherence using semantic memory.

Step-wise Information Flow:

1. **User Input (Presentation Layer)**: The flow begins when the user provides input through the **Streamlit user interface**, such as a seed idea, genre, tone, or stage selection.

2. **Data Transfer to Application Layer**: The input data is captured and forwarded to the **Content Pipeline**, where it is structured for further processing.

3. **Context Retrieval (Memory Layer)**: The Content Pipeline sends a query to the **FAISS vector store** to retrieve top-k semantically relevant embeddings based on the current input.

4. **Context Integration**: The retrieved contextual data is integrated with the user input to enrich the prompt, ensuring continuity and coherence across stages.

5. **Prompt Transmission to AI Layer**:The structured prompt is passed to the **LLM Client**, which forwards it to the **Generative AI model (e.g., Gemini API)**.

6. **AI Processing and Response Generation**: The AI model processes the input and generates structured content based on the prompt and contextual information.

7. **Response Handling (Application Layer)**: The generated output is received by the Content Pipeline, where it is formatted and prepared for both display and storage.

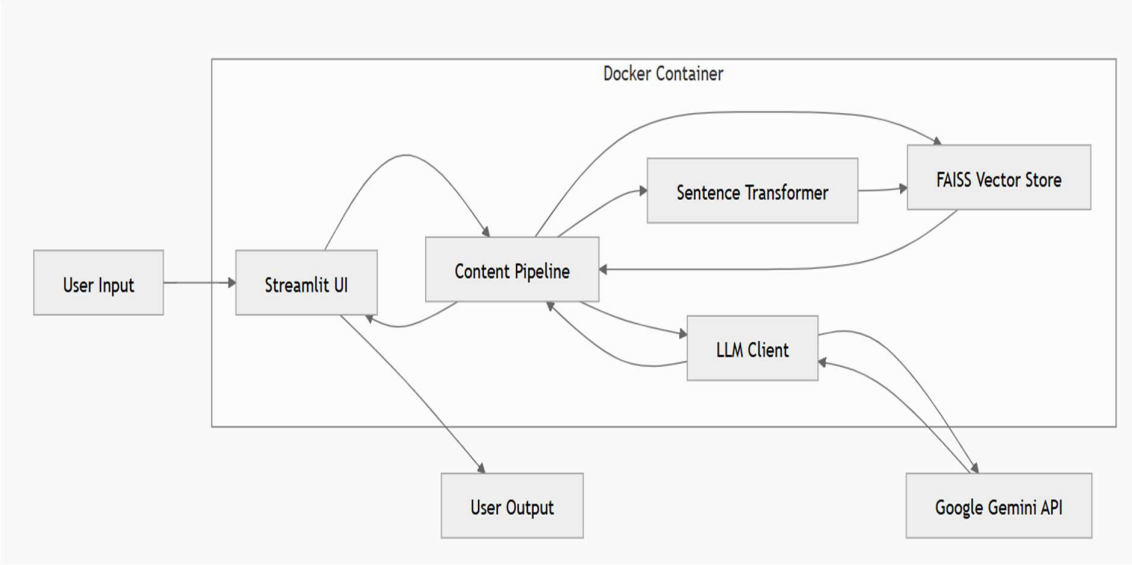
8. **Embedding and Storage (Memory Layer)**: The output is converted into vector embeddings using a **Sentence Transformer**, and stored in the **FAISS database** along with metadata such as stage and timestamp.

9. **State Update and Data Propagation**: The processed data is passed back to the UI layer, where the application state is updated accordingly.

10. **User Output (Presentation Layer)**: The final output is rendered on the UI in a structured and user-friendly format.

11. **Containerized Flow Execution**: All data exchanges and processing steps occur within the **Docker container**, ensuring:

- Consistent data handling
- Environment independence
- Reliable execution across systems



2.4 Components Design

The **Dockerized Entertainment React Service** is designed using a modular approach, where each component has clearly defined responsibilities. This ensures separation of concerns, ease of maintenance, and scalability. The system primarily consists of components across the **presentation layer, application layer, AI layer, memory layer, and deployment layer**.

2.4.1 Streamlit UI (Presentation Layer)

Responsibilities:

- Accept user inputs such as idea, genre, tone, and stage selection
- Display generated content in a structured and user-friendly format
- Provide options for full pipeline execution or stage-wise generation
- Enable regeneration of outputs
- Manage session state

This component acts as the **user interaction interface** and does not directly communicate with the AI or memory layers.

2.4.2 Content Pipeline (Application Layer)

Responsibilities:

- Orchestrate the execution of all content generation stages
- Construct structured prompts using user input and contextual memory
- Manage workflow using functions such as:
    - run_stage()
    - run_full_pipeline()
- Coordinate communication between UI, AI layer, and memory layer

This is the **core controller** of the system and ensures a smooth and structured execution flow.

2.4.3 LLM Client (AI Integration Layer)

Responsibilities:

- Connect to the Generative AI service (e.g., Google Gemini API)
- Apply system-level constraints and prompt formatting
- Handle API requests, retries, and rate limits
- Return structured AI-generated responses

This component abstracts the interaction with external AI services, ensuring flexibility and loose coupling.

2.4.4 Vector Store – FAISS (Memory Layer)

Responsibilities:

- Store generated content as vector embeddings
- Maintain metadata such as stage, timestamp, and content ID
- Retrieve top-k semantically similar content for context
- Enable context-aware and consistent content generation

This component ensures **semantic continuity across multiple stages**.

2.4.5 Sentence Transformer (Embedding Model)

Responsibilities:

- Convert textual content into numerical vector embeddings
- Support semantic similarity search in FAISS
- Enable efficient retrieval of relevant context

This component works closely with the vector store to enhance AI output quality.

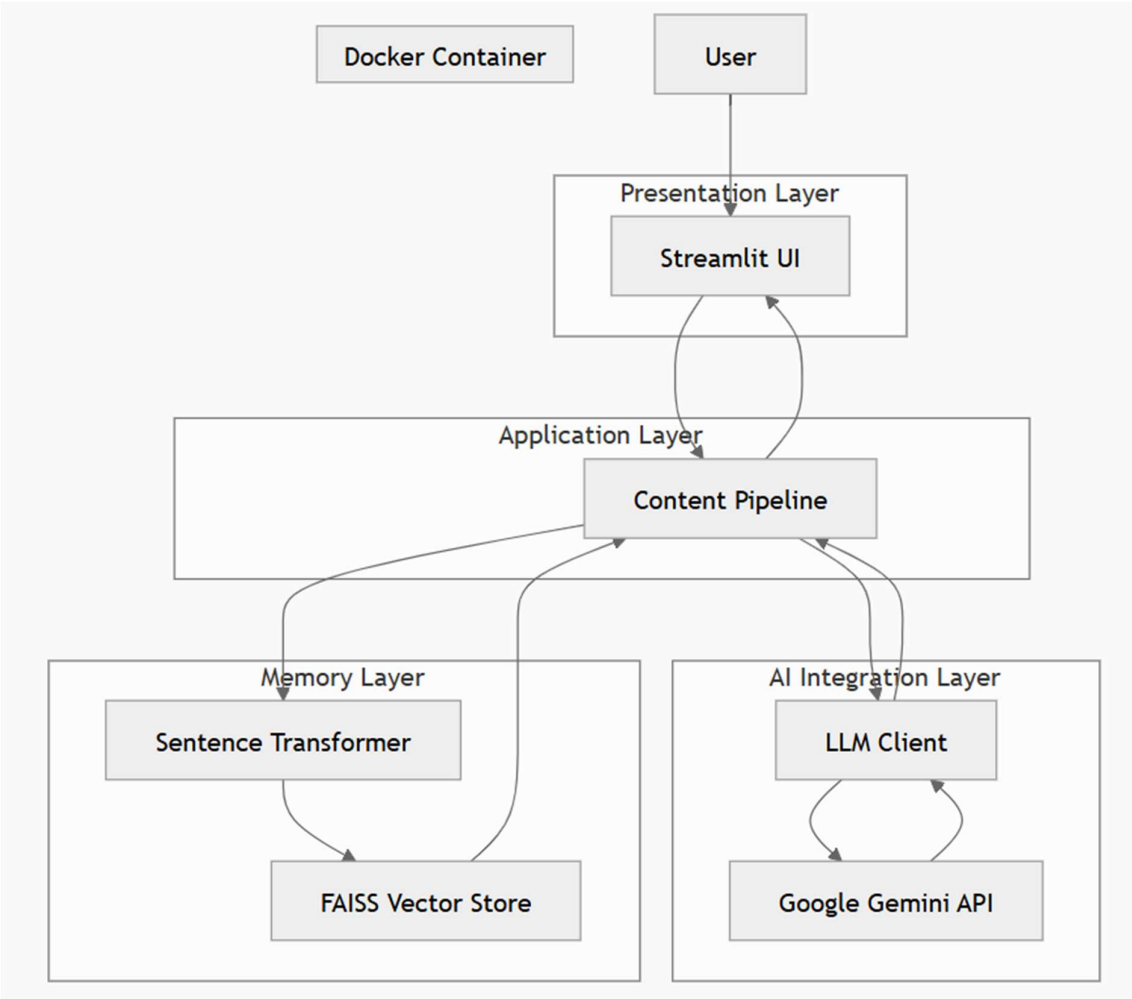
2.4.6 Docker Container (Deployment Layer)

Responsibilities:

- Package the entire application including all components and dependencies
- Provide an isolated runtime environment

- Ensure consistent execution across different platforms
- Simplify deployment and scaling

This component is the **key addition in this project**, transforming the system into a deployable solution.



2.5 Key Design Considerations

2.5.1 Modularity: The system is divided into independent layers (UI, content pipeline, AI, and memory), enabling easy development and maintenance.

2.5.2 Reusability: The existing Generative AI pipeline is reused without major changes, reducing development effort and ensuring consistency.

2.5.3 Scalability: The use of Docker and a decoupled architecture allows the system to scale and supports future cloud-based deployment.

2.5.4 Deployment Consistency: Docker ensures a uniform environment across all platforms by packaging all dependencies together.

2.5.5 Performance Optimization: Efficient vector search (FAISS) and controlled AI API calls help maintain good system performance.

2.5.6 Context Consistency: Semantic embeddings and similarity search ensure coherent and context-aware content generation.

2.5.7 Maintainability: The modular structure makes the system easy to understand, update, and debug.

2.5.8 Extensibility: The system can be extended with new features, models, or storage solutions in the future.

2.6 API Catalogue

The system interacts with external AI services and internal processing functions to perform content generation and data handling. The following APIs and core functions are used:

2.6.1 Content Generation API (LLM API)

- **Service:** Google Gemini API
- **Method:** POST

- **Description:** Generates structured entertainment content based on user input and constructed prompts
- **Input:** Prompt (idea, genre, tone, context)
- **Output:** AI-generated textual content

2.6.2 Run Stage Function

- **Function:** run_stage(stage_name)
- **Description:** Executes a specific stage of the content generation pipeline (e.g., concept, logline, outline)
- **Input:** Stage name, user input, context
- **Output:** Generated content for the selected stage

2.6.3 Full Pipeline Execution

- **Function:** run_full_pipeline()
- **Description:** Executes all stages sequentially to generate complete structured content
- **Input:** User input parameters
- **Output:** Multi-stage generated content

2.6.4 Context Retrieval Function

- **Function:** retrieve_context()
- **Description:** Fetches top-k relevant embeddings from FAISS for maintaining context
- **Input:** Query embedding
- **Output:** Relevant contextual data

2.6.5 Embedding Generation Function

- **Function:** generate_embedding()

- **Description:** Converts generated text into vector embeddings using a Sentence Transformer model
- **Input:** Text data
- **Output:** Numerical embedding vector

3. Data Design

The data design of the system focuses on efficiently handling **user inputs, generated content, and semantic embeddings** required for context-aware content generation. The system uses a combination of structured data and vector representations to ensure smooth processing and retrieval.

3.1 Data Model

The data model is designed to support the **multi-stage content generation pipeline** and maintain contextual continuity across stages.

1. Generated Content Data

This represents the textual output generated at each stage of the pipeline.

Attributes:

- **Content ID:** Unique identifier for each generated output
- **Stage:** Type of content (Concept, Logline, Outline, Character, Scene)
- **Text:** Generated content
- **Timestamp:** Time of creation
- **Version:** Tracks regenerated outputs

2. User Input Data

This captures the initial input provided by the user.

Attributes:

- **Idea:** Seed input for content generation

- **Genre:** Type of content (e.g., Action, Drama)
- **Tone:** Style or mood of content

3. Embedding Data (Vector Data)

This represents the numerical vector form of generated content used for semantic search.

Attributes:

- **Embedding Vector:** Numerical representation of text
- **Content ID:** Reference to generated content
- **Stage:** Stage of generation
- **Metadata:** Additional information (genre, tone, timestamp)

4. Storage Mechanism

- **FAISS Vector Database:**
  Stores embedding vectors for fast similarity search
- **Metadata Storage (JSON / In-Memory):**
  Stores content details and associated attributes

3.2 Data Access Mechanism

The system follows a structured data access mechanism to ensure efficient interaction between the user interface, content generation pipeline, AI services, and memory layer. Data access is controlled and managed through the application layer to maintain consistency and security.

1. Layered Access Approach

- **Presentation Layer (Streamlit UI):**
  Captures user input and displays generated output. It does not directly access internal data storage or AI services.

- **Application Layer (Content Pipeline):**
  Acts as the central controller for all data operations. It manages the flow of data between the UI, AI layer, and memory layer.

2. Read Operations

- Retrieves contextual data from the **FAISS vector store** using similarity search
- Fetches stored embeddings and metadata required for prompt construction
- Ensures relevant context is used for content generation

3. Write Operations

- Stores generated content after each stage
- Converts content into embeddings and stores them in FAISS
- Maintains associated metadata such as stage, timestamp, and identifiers

4. AI Data Access

- The Content Pipeline constructs prompts and sends them to the **LLM API**
- Receives generated responses and processes them for further use

5. Controlled Access

- Direct access to the vector store and AI services is restricted to the Content Pipeline
- The UI does not interact directly with internal data layers
- This ensures better data integrity and system security

3.3 Data Retention Policies

The system follows simple and efficient data retention practices to manage generated content and embeddings while maintaining performance and security.

1. Retention Scope

The system retains the following data:

- Generated content from each stage
- Corresponding vector embeddings
- Metadata such as stage, timestamp, and content ID

2. Retention Duration

- Data is stored **temporarily during application usage**
- For extended sessions, data may persist in memory for context continuity
- Long-term storage is not mandatory in the current implementation

3. Storage Management

- Embeddings are stored in the **FAISS vector store** for fast retrieval
- Metadata is maintained in **in-memory structures or lightweight storage (JSON)**
- Old or unused data can be cleared to maintain system efficiency

4. Data Cleanup

- Temporary data is cleared when the session ends
- Redundant or outdated embeddings can be removed periodically
- Ensures optimized memory usage and performance

5. Security Considerations

- No sensitive user data is permanently stored
- Only necessary input and generated content are retained
- Data access is restricted to the application layer

3.4 Data Migration

The current system is designed as a **lightweight and mostly stateless application**, therefore extensive data migration is not required in its present form.

1. Current Approach

- Data such as generated content and embeddings is stored temporarily
- No permanent database is used for long-term storage
- The system relies on in-memory storage and FAISS for runtime operations

2. Migration Requirement

- Since there is no persistent database, **data migration is minimal or not applicable**
- Each session operates independently without dependency on previous data

4. Interfaces

The system interacts with users and external services through clearly defined interfaces, ensuring smooth communication between different components.

4.1 User Interface (UI)

- Built using **Streamlit**
- Allows users to:
    - Enter input (idea, genre, tone)
    - Select generation stages
    - View structured AI-generated content
    - Regenerate outputs if required

The UI provides a simple and interactive way for users to interact with the system.

4.2 Application Interface

- Acts as a bridge between the UI and backend processing

- Managed by the **Content Pipeline**
- Handles:
  - Input processing
  - Prompt construction
  - Data flow between components

4.3 AI Service Interface

- Integrates with **Generative AI APIs (e.g., Gemini)**
- Uses API-based communication for content generation
- Input: Structured prompts
- Output: Generated textual content

4.4 Memory Interface

- Connects the application layer with the **FAISS vector store**
- Handles:
  - Storage of embeddings
  - Retrieval of contextual data

5. State and Session Management

The system manages state and session data to ensure smooth user interaction and continuity during content generation.

1. State Management

- Managed using **Streamlit's session state**
- Stores:
  - User inputs (idea, genre, tone)
  - Generated outputs at each stage

  - Selected generation stage or mode
- Enables dynamic updates of the UI based on user actions

2. Session Management

- Each user interaction runs within a **single session**
- Data is maintained only for the duration of the session
- Supports:
  - Multi-stage content generation
  - Regeneration of outputs
  - Continuity during user interaction

3. Data Persistence

- No long-term session storage is implemented
- Data is cleared once the session ends or the application is restarted

6. Caching

The current system does not implement a dedicated caching mechanism. However, certain components contribute to efficient data retrieval and performance.

1. Implicit Caching through FAISS

- The **FAISS vector store** enables fast retrieval of relevant embeddings
- Acts as a form of efficient lookup for previously generated content
- Reduces the need for repeated processing

2. In-Memory Data Usage

- Generated outputs are temporarily stored during the session

- Helps avoid redundant computations within the same session

3. Future Enhancements

- Integration of caching systems like **Redis**
- API response caching to reduce repeated LLM calls
- Improved performance for large-scale usage

7. Non-Functional Requirements

The system is designed to meet essential non-functional requirements to ensure secure, efficient, and reliable operation in a containerized environment.

7.1 Security Aspects

- **API Key Protection:**
  Access to the Generative AI service is secured using API keys, which are not exposed in the user interface.
- **Controlled Access:**
  Only the Content Pipeline interacts with internal components such as the AI service and vector store.
- **No Sensitive Data Storage:**
  The system does not store personally identifiable or sensitive user data.
- **Secure Communication:**
  API interactions can be conducted over secure protocols (HTTPS) to prevent data interception.

7.2 Performance Aspects

- **Efficient Vector Search:**
  FAISS enables fast retrieval of relevant context using similarity search.
- **Optimized AI Calls:**
  Structured prompts and controlled API usage reduce unnecessary requests and latency.

- **Lightweight Processing:**
  Use of efficient embedding models ensures quick computation.
- **Containerized Execution:**
  Docker provides a stable and optimized runtime environment, improving consistency and performance.

8. References (IEEE Format)

[1] Google, "Google Gemini API Documentation." [Online]. Available: https://ai.google.dev

[2] Facebook AI Research, "FAISS: A Library for Efficient Similarity Search." [Online]. Available: https://github.com/facebookresearch/faiss

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *arXiv preprint arXiv:1908.10084*, 2019.

[4] Streamlit Inc., "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io

[5] Docker Inc., "Docker Documentation." [Online]. Available: https://docs.docker.com

[6] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral dissertation, University of California, Irvine, 2000.

[7] OpenAI, "Introduction to Large Language Models." [Online]. Available: https://platform.openai.com/docs

[8] Hugging Face, "Transformers Library Documentation." [Online]. Available: https://huggingface.co/docs

[9] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," *arXiv preprint arXiv:1301.3781*, 2013.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.

[11] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.